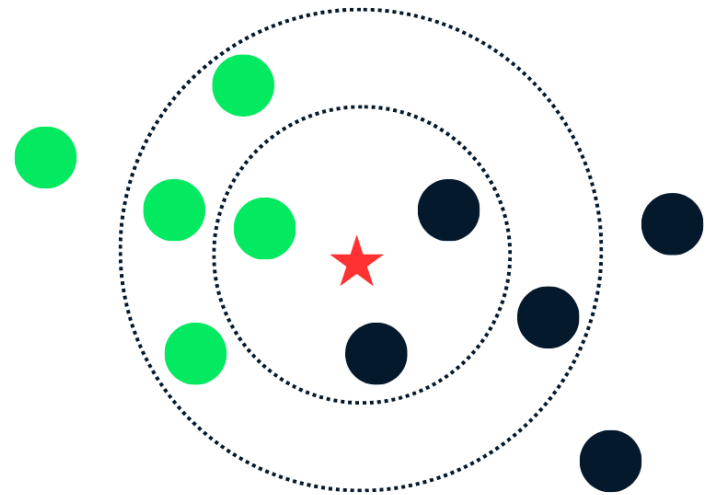


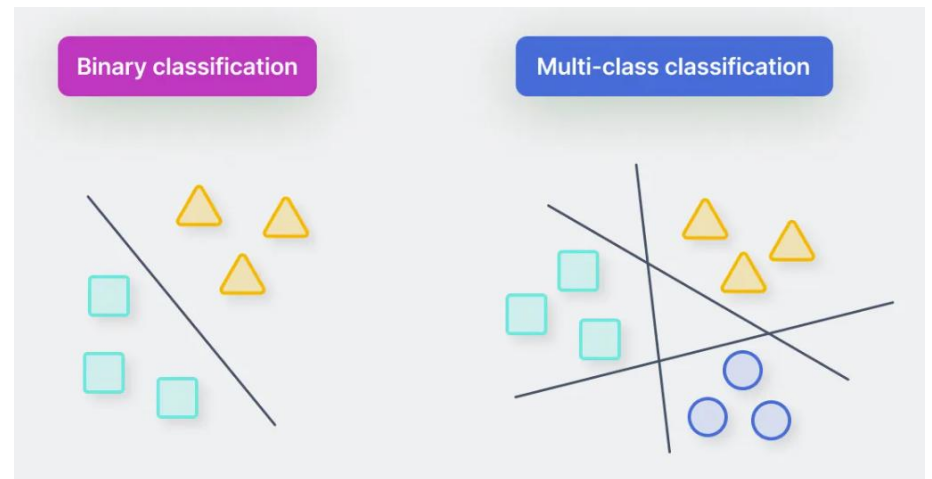
K-Nearest Neighbour (k NN) Classifier



Outline

- Classification
- K-Nearest Neighbour (kNN) Classifier

Classification

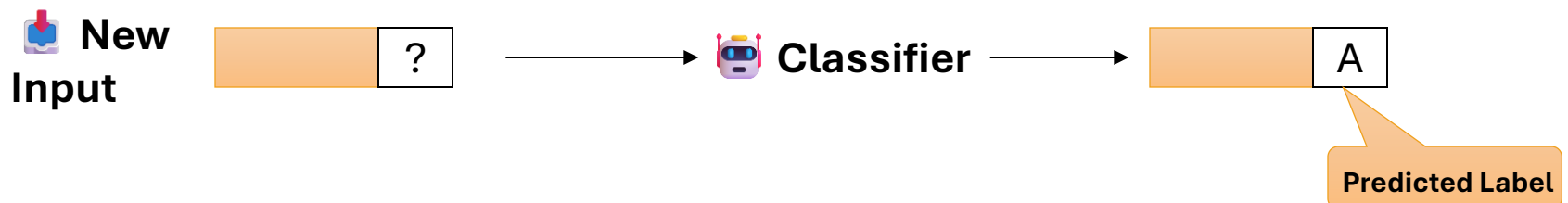


Classification Models

-  **Goal:** Predict the class label of an input instance using supervised learning
- Supervised learning two-stage process:
 - Given a **labeled** dataset containing features and their class labels, use a  Learning Algorithm to train a  **Classifier** that maps features to class labels



- Given a **new input** with unknown label, use the trained  **Classifier** to assign the most likely class



Classification Examples

- **Spam Detection (Binary)**

Classifies emails as *spam* or *not spam* by learning patterns from text features and sender metadata.

- **Sentiment Analysis (Multi-class)**

Predicts whether reviews or social-media posts express *positive*, *negative*, or *neutral* sentiment.

- **Medical Diagnosis (Binary)**

Uses patient symptoms and clinical history to classify the likelihood of a specific condition.

- **Credit Risk Assessment (Multi-class)**

Categorizes applicants into *low*, *medium*, or *high-risk* using features such as credit score, income, and debt ratios.

- **Image Recognition (Multi-class)**

Classifies images into categories (e.g., *cat*, *dog*, *bird*) based on learned visual features.

Classification Algorithms



K-Nearest Neighbors (KNN)

Instance-based classifier that predicts using distance to nearby examples.



Decision Trees & Random Forests

Tree-structured models that learn hierarchical decision rules.



Logistic Regression

Linear classifier that estimates class probabilities using a logistic function.



Naive Bayes

Probabilistic model assuming conditional independence between features.



Support Vector Machine (SVM)

Maximizes class-separation margin using an optimal hyperplane.



Artificial Neural Networks (ANN)

Multi-layer networks capable of learning complex nonlinear decision boundaries.

ML Metrics: Key Factors for Model Quality

Accuracy

Proportion of correctly classified instances out of all predictions made

Performance

- Training time required to build the model
- Inference time required to generate predictions

Interpretability

- How easily model decisions can be understood

Robustness

Ability to handle noise, missing values, and imperfect data

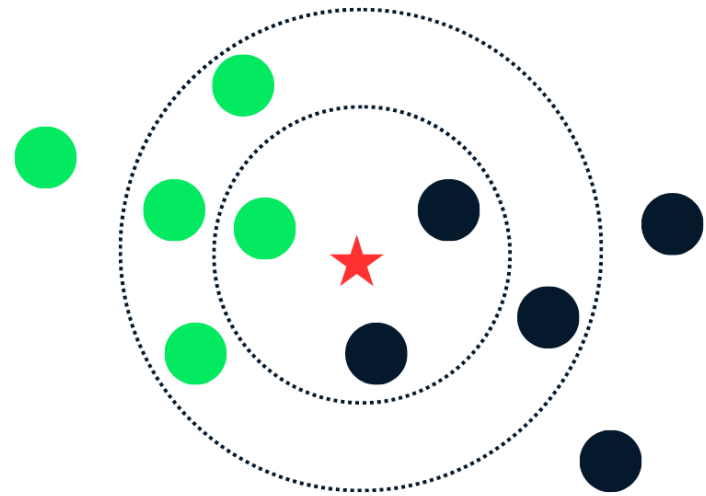
Scalability

Capability to operate efficiently on large datasets

Model Complexity

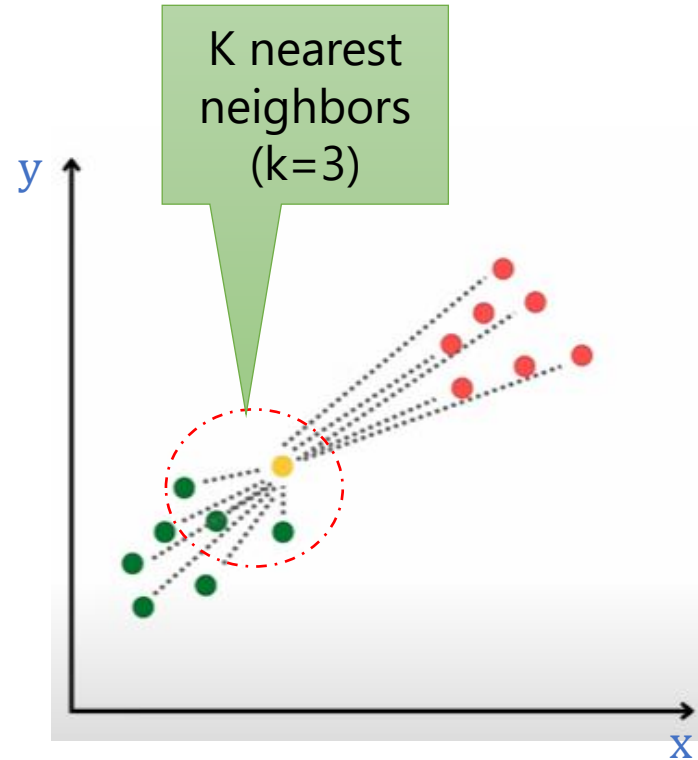
Size and compactness of the learned representation (e.g., depth of trees, number of rules)

K-Nearest Neighbour (k NN) Classifier



kNN Algorithm steps

- kNN predicts the label of a new instance based on the labels of the most **similar** training examples.
- Given a data point that we want to classify:
 1. **Compute the distance** between the new instance and all training examples
 2. **Locate the k nearest neighbors** with the smallest distances
 3. For classification: Assign the **majority class** among the k neighbors
 3. For regression: Compute the **average value** of the k neighbors



K-Nearest Neighbour Model

- **Lazy Learner:**
 - It does not build a model during training
 - All computation happens at the time of classification => ideal where data is constantly changing
- **Instance-based** learning, i.e., **no explicit model**, but rather classified based on training instances
- **Non-parametric** algorithm, i.e., **no assumptions** about the underlying data distribution
- **Similarity-based** learning:
 - Assumes that similar instances have similar labels
 - Classifies new examples based on their closest neighbors

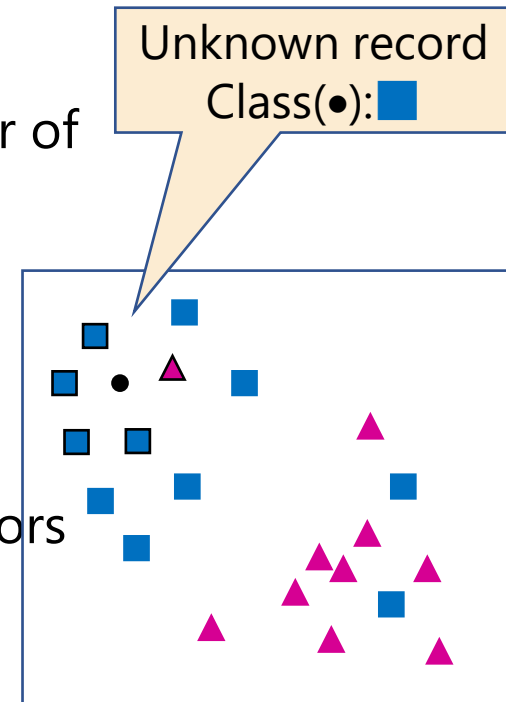
Requirements & Steps

- **Requirements:**

- The set of training instances
- **Distance Metric** to compute distance between instances
- The value of the **hyperparameter k** , i.e., the number of nearest neighbors to consider

- **To predict an unknown instance:**

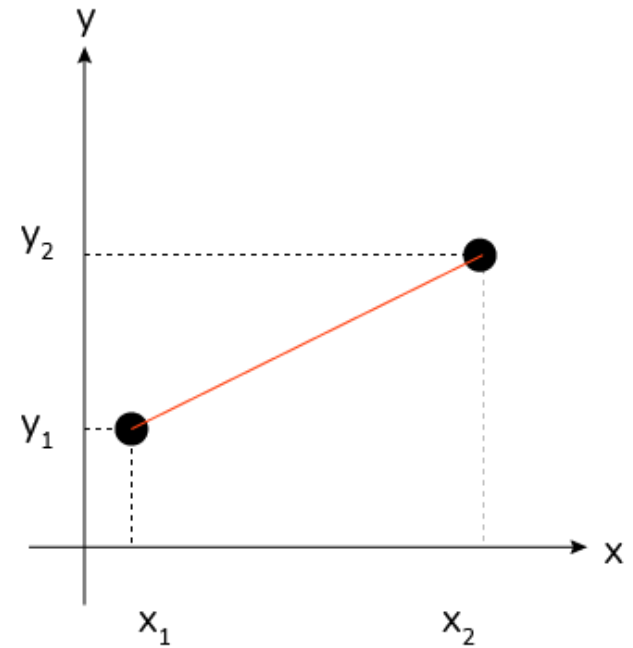
- Compute distances to all training instances
- Locate **k** nearest neighbors
- **Classify using majority vote** among the k neighbors
- In the case of a **tie**:
 - Use **odd k** (doesn't solve multi-class)
 - Choose the class with the **closest neighbor**
 - Pick the class **dominant** in the entire dataset
 - **Decrease k** by 1 until you **break** the tie



Euclidean Distance

- **Euclidean Distance** is a measure of the **straight-line distance** between two points (x_1, y_1) and (x_2, y_2) in a two-dimensional space

It's derived from the **Pythagorean** theorem



$$\text{Distance} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

- Numerical measure of **Dissimilarity/Distance**
 - How **different** are two data objects
 - Lower when objects are more alike

Euclidean Distance for data objects with multiple features

- When dealing with instances (data objects) with multiple n features, the Euclidean distance between two instances p and q with features $(p_1, p_2, \dots, p_n)$ and $(q_1, q_2, \dots, q_n)$ respectively, is given by:
 - the square root of the sum of the squared differences between corresponding feature values

$$\text{Distance} = \sqrt{\sum_{i=1}^n (q_i - p_i)^2}$$

- p_i and q_i represent the values of the i^{th} feature for data objects p and q respectively
- n is the number of features



Distance Metrics

- Euclidean distance:

$$d_{Eucl}(a, b) = \sqrt{\sum_{k=1}^m (a_k - b_k)^2}$$

Measures
straight-line
geometric distance

- Manhattan distance:

$$d_{Manh}(a, b) = \sum_{k=1}^m |a_k - b_k|$$

Sum of absolute
differences

- Minkowski distance:

$$d_{Mink}(a, b) = \sqrt[p]{\sum_{k=1}^m |a_k - b_k|^p}$$

Family of metrics

- When $p = 1$, the Minkowski distance is the Manhattan distance
- When $p = 2$, the Minkowski distance is the Euclidean distance
- Although there are infinite number of Minkowski-based distance metrics to choose from, Euclidean distance and Manhattan distance are the most used ones

When to Use Each Distance Metric



Euclidean Distance ($p = 2$)

- Features are continuous and well-scaled
- Data tends to form spherical clusters
- You want sensitivity to large differences

Typical for: image data, sensor signals, geometric data.



Manhattan Distance ($p = 1$)

- Data changes one feature at a time, not in combined directions
- You want reduced sensitivity to outliers
- High-dimensional datasets where Euclidean becomes unstable

Typical for: text embeddings, tabular categorical-converted-to-numeric data.



Minkowski ($p > 2$ or fractional p)

- You want to experiment with different geometries
- You need a metric between Manhattan and Euclidean
- You want to tune p as a hyperparameter (rare but sometimes useful)

Similarity Between Binary Vectors using Simple Matching Coefficient (SMC)

- SMC quantifies the similarity between two **binary** vectors p & q (e.g., gender, yes/no, presence/absence)
 - **SMC ranges from 0** (no similarity) **to 1** (perfect similarity, all matches)

$$SMC = \frac{M_{11} + M_{00}}{M_{11} + M_{00} + M_{01} + M_{10}}$$

M₁₁: Counts positions where **both vectors have 1**

M₀₀: Counts positions where **both vectors have 0**

M₀₁: Counts positions where **p = 0** and **q = 1**

M₁₀: Counts positions where **p = 1** and **q = 0**

- In the case of **multiple features**, the formula is applied to each feature individually, and then averaged across all n features:

$$SMC = \frac{1}{n} \sum_{i=1}^n \frac{M_{11}^{(i)} + M_{00}^{(i)}}{M_{11}^{(i)} + M_{00}^{(i)} + M_{01}^{(i)} + M_{10}^{(i)}}$$

Similarity Between Binary Vectors using Jaccard Coefficient

- Jaccard Coefficients is used to quantify the similarity between two **binary** vectors

$$J = \frac{\text{number of 11 matches}}{\text{number of not-both-zero attribute values}}$$

$$J = \frac{M_{11}}{M_{01} + M_{10} + M_{11}}$$

- SMC / Jaccard ranges from 0 to 1, where:
 - 0 indicates no similarity between the instances
 - 1 indicates perfect similarity between the instances
 - Higher SMC values suggest greater similarity between the instances, while lower values suggest greater dissimilarity

SMC versus Jaccard - Example

$$p = 1, 0, 1, 0, 1$$

$$q = 1, 1, 0, 0, 1$$

$M_{11} = 2$ (the number of attributes where p is 1 and q is 1)

$M_{00} = 1$ (the number of attributes where p is 0 and q is 0)

$M_{01} = 1$ (the number of attributes where p is 0 and q is 1)

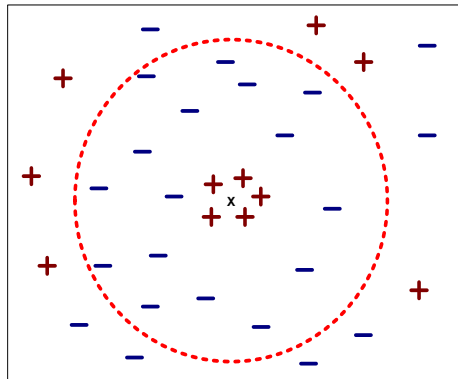
$M_{10} = 1$ (the number of attributes where p is 1 and q is 0)

$$\text{SMC} = \frac{M_{11} + M_{00}}{M_{11} + M_{00} + M_{01} + M_{10}} = \frac{2 + 1}{2 + 1 + 1 + 1} = 0.6$$

$$J = \frac{M_{11}}{M_{01} + M_{10} + M_{11}} = \frac{2}{2 + 1 + 1} = 0.5$$

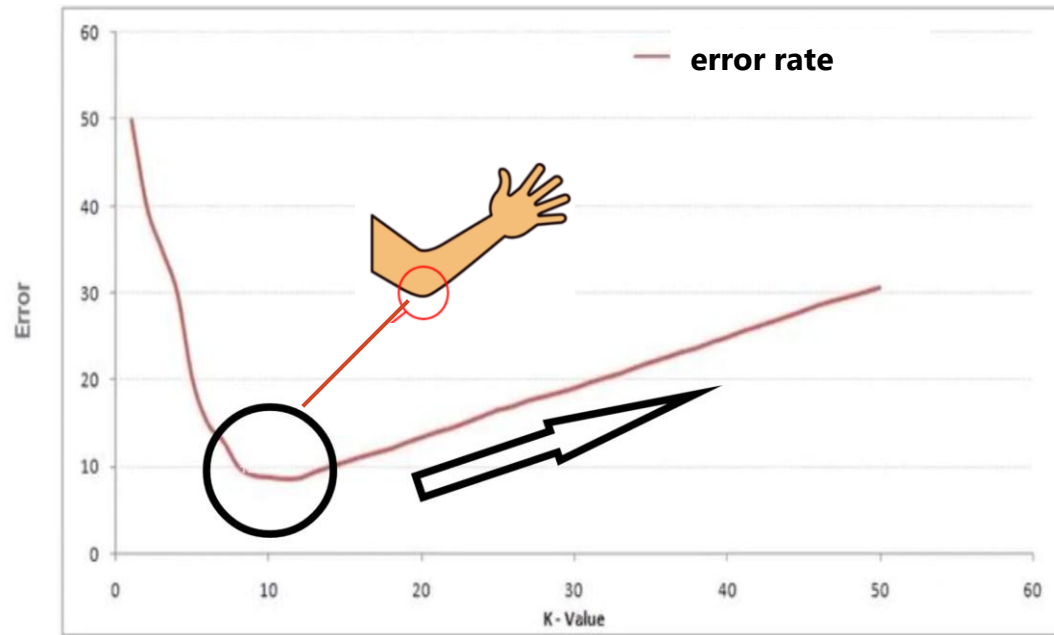
Choosing the value of k

- **k too small → Overfitting = poor generalization (High Variance)**
Model becomes too sensitive to noise; predictions change a lot with small changes in the data
- **too large → Underfitting (High Bias)**
Neighborhood becomes too broad; the model oversimplifies patterns and mixes points from different classes
- **Goal:**
Select a k that balances **bias and variance** for optimal classification performance.



The Elbow Method

1. Train kNN with a range of k values
2. Compute the error rate for each k
3. Plot error vs. k



- Identify the "elbow" point on the curve, where the **error rate stops decreasing sharply** and starts to level off
- This elbow indicates the **best trade-off** between overfitting (small k) and underfitting (large k).
- **Choose k at the elbow point** as the optimal number of neighbors

K-NNs Properties

- 🐢 **Lazy (Instance-Based) Learner**

kNN does **not** build an explicit model during training

- 🧩 **Simple to Implement**

Easy to understand and apply; often a strong baseline method.

- 📏 **Requires Feature Scaling**

Distance can be dominated by features with large numeric ranges (e.g., height 1.5–1.8 m, weight 90–300 lb, income 10k–500k QR)

- ⚙️ **Minimal Tuning**

Only a few choices matter: **k** and the **distance metric**.

- 👍 **Surprisingly Effective**

Often performs well—worth trying first on many problems.

- ⌚ **Expensive at Prediction Time**

Classifying a new instance requires scanning the **entire dataset**, which becomes costly for large datasets.

KNN Limitations



Slow on large datasets because it computes distances to every training point.



Performs poorly in high dimensions since accuracy degrades as the number of features grows.



Sensitive to the distance metric; different metrics can produce different results



Biased toward majority class when data is imbalanced



Requires handling missing values; simple imputation (e.g., replacing with the mean) can distort distances.